Trabalho 3 (T3): Divisão e conquista com MPI

Unidade 04 | Grupos de até 3 integrantes

O trabalho tem quatro momentos: a apresentação deste enunciado, duas aulas de acompanhamento em laboratório, a entrega no Moodle (véspera da apresentação) e a **apresentação para a turma**. As datas estão no cronograma do Moodle. Junto deste enunciado estão publicados o arquivo `sequencial.c`, com as rotinas da seção 2, e o artigo de Radenski citado na seção 4, com o link para o código dele.

---

1. Objetivo

Implementar, usando a biblioteca MPI, uma versão paralela seguindo o modelo **divisão e conquista** de um programa adequado a esse modelo, executá-la no cluster Atlântica do LAD com até 2 nós e **analisar o desempenho** obtido. Deve ser medida também uma versão em que o trabalho é **balanceado na árvore**. Como nos trabalhos anteriores, o foco é a análise: explicar o que causou os números, não repetir em texto o que o gráfico já mostra.

2. O problema: ordenação de um vetor

O problema sugerido é a ordenação de um grande vetor. Outro problema adequado ao modelo pode ser usado, se validado pelo professor no primeiro acompanhamento.

O processo raiz da árvore verifica se o vetor inicial será **dividido** ou **conquistado**. A verificação é uma comparação do tamanho do vetor com um valor fixo δ. Se for dividido, cada filho da árvore binária recebe uma metade do vetor e, por sua vez, verifica se vai conquistar ou dividir de novo. Quando uma folha decide que o vetor já está pequeno o bastante, ela o ordena com o **Bubble Sort** e o devolve ao pai. O pai **intercala** os dois vetores recebidos e devolve o resultado ao seu pai, até que a raiz tenha o vetor original ordenado.

Por exemplo, com um vetor de 40 elementos e $\delta = 10$, a raiz divide em duas metades de 20, cada filho divide de novo em metades de 10, e as quatro folhas conquistam: a árvore desce 2 níveis e usa 7 processos (Figura 1). Em geral, δ não é um parâmetro a ajustar: ele sai do número de processos, para que a árvore ocupe exatamente os P processos alocados. Com $P = 2^k - 1$ processos há $(P + 1)/2$ folhas, e $\delta = \lceil 2n/(P + 1) \rceil$. O arredondamento é para cima: quando a divisão não é exata, as folhas recebem pedaços que diferem em um elemento, e com δ truncado o pedaço maior tentaria se dividir de novo, para filhos que não existem.

O vetor começa em **ordem decrescente**, o pior caso do Bubble Sort. O `sequencial.c` traz o Bubble Sort, a rotina de intercalação e duas versões sequenciais:

- `./sequencial n` ordena o vetor inteiro com o Bubble Sort;
- `./sequencial n delta` percorre a **mesma árvore** de divisões da versão paralela, em um só processo: divide enquanto o pedaço passa de δ, ordena as folhas com o Bubble Sort e intercala na volta.

Com 3 processos (uma raiz e dois filhos), o tempo de ordenação cai para cerca de 1/4 do tempo com um processo. O ganho é maior que 3 porque o Bubble Sort é quadrático: ordenar duas metades custa metade de ordenar o vetor inteiro. Parte do ganho vem do paralelismo e parte da mudança de algoritmo, e a segunda versão sequencial serve para separar as duas.

Ferramentas de IA estão liberadas nas mesmas condições dos trabalhos anteriores: o grupo deve saber explicar o código linha a linha, e o relatório deve dizer como a ferramenta foi usada.

3. A versão paralela

Cada processo executa o mesmo programa; o anexo traz o pseudocódigo da versão da Figura 1. A implementação precisa atender aos requisitos abaixo:

1. **SPMD.** Um único executável; o papel de cada processo (raiz, interno, folha), seu pai e seus filhos saem do rank.

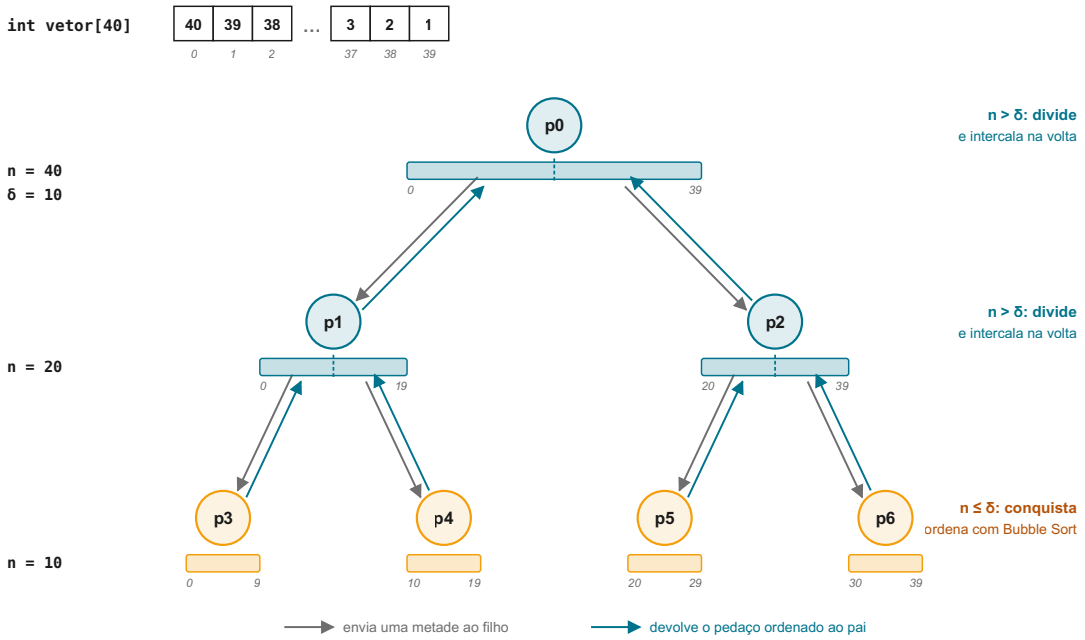


Figura 1: Divisão e conquista na ordenação de um vetor de 40 elementos com $\delta = 10$ e 7 processos.

2. **O tamanho vem na mensagem.** O filho descobre quantos elementos recebeu com `MPI_Get_count` (ou `MPI_Probe` antes de alocar).
3. **O pai envia as duas metades antes de esperar qualquer resultado**, para que os dois filhos trabalhem ao mesmo tempo.
4. **O tamanho do vetor e** δ (ou o número de níveis) são lidos da linha de comando.

4. Balanceamento de carga

Na versão da Figura 1, um processo que dividiu fica **parado** até os filhos responderem: só as folhas conquistam. Analise esse desbalanceamento e como ele pode ser compensado de duas formas:

Sem alterar o código: execute com mais processos do que núcleos, acima do HT (63, 127, 255, 511 e 1023 processos). Os processos que esperam deixam o núcleo livre, e o escalonador do sistema operacional faz o balanceamento. Isso só acontece se o processo parado em `MPI_Recv` de fato liberar o núcleo. Com o `srun`, o Open MPI faz **espera ativa** por padrão: o processo verifica a chegada da mensagem em laço e continua ocupando a thread que as folhas poderiam usar. Com a variável de ambiente `OMPI_MCA_mpi_yield_when_idle=1`, ele cede o núcleo enquanto espera.

Alterando o código: ao dividir, o processo envia uma metade para um filho e **fica com a outra**, em vez de entregar as duas e esperar (a árvore abre para um dos lados apenas). Assim, os processos internos também conquistam.

O artigo de Radenski distribuído no Moodle descreve uma implementação dessa ideia, e o código dele é público (licença GPL): `mpi_mergesort.c`. Ele pode ser usado como ponto de partida, com a origem citada no relatório e três condições:

1. **As folhas ordenam com o Bubble Sort**, e o vetor começa em ordem decrescente, como na versão original. O código de Radenski ordena as folhas com outro algoritmo e parte de um vetor aleatório; sem a troca, a comparação entre as duas versões mede a diferença entre algoritmos de ordenação, não o balanceamento.
2. **O tempo é medido com `MPI_Wtime()`**, na mesma região da versão original (seção 5).

3. **O grupo sabe explicar cada chamada MPI do código adaptado**, como qualquer código que entrega: por que o ajudante de um processo é `rank + 2^nível`, o que fazem o `MPI_Isend` e o `MPI_Request_free` e por que o buffer enviado pode ser reaproveitado.

Nesse esquema, cada divisão entrega metade do pedaço a um novo processo. Com P potência de 2, todos terminam com n/P elementos; com $P = 2^k - 1$, não.

5. Ambiente e regras de medição

Todas as medições, inclusive a sequencial, são feitas **no cluster Atlântica**. Cada nó tem 2 processadores Xeon E5520 de 4 núcleos físicos, com Hyper-Threading (HT): **8 núcleos físicos e 16 threads de hardware por nó**. O enunciado do T2 indicava o dobro; o `lscpu` dos grupos mostrou estes números. Confirme com `lscpu` e registre no relatório.

1. **Nós exclusivos.** Rode com `--exclusive`, sempre nos mesmos **2 nós**, e prefira jobs em lote (`sbatch`). Diga como os processos foram distribuídos entre os nós. Rodar mais processos que threads de hardware exige liberar o Slurm para isso (`--overcommit`).
2. **Duas referências sequenciais**, compiladas com as mesmas flags de otimização da paralela e executadas em um nó do Atlântica: T_1, o Bubble Sort do vetor inteiro, e T_1^δ, a mesma árvore em um só processo, medida para cada δ usado nas versões paralelas. Na versão balanceada, as folhas podem ficar com tamanhos diferentes, e um δ único deixa de reproduzir a árvore. O relatório diz qual árvore a referência percorre e mostra que ela faz o mesmo trabalho da versão paralela; se preciso, adapte o `sequencial.c`. Dimensione o vetor para que T_1 fique **entre 30 e 60 segundos**, e informe o tamanho no relatório.
3. **Região medida:** com `MPI_Wtime()` na raiz, do início da distribuição do vetor até a raiz ter o vetor inteiro ordenado. A inicialização do vetor fica de fora nas duas versões.
4. **Equivalência registrada:** toda execução verifica que o vetor final está ordenado e é o mesmo da versão sequencial, e imprime o resultado da verificação.
5. **10 repetições** por configuração; reporte mediana, mínimo e máximo em todas as tabelas.

6. Medições exigidas

1. As versões sequenciais: T_1 e T_1^δ para cada δ usado.
2. A versão paralela com **1, 3, 7, 15 e 31 processos** em 2 nós exclusivos. Com 2 nós há 16 núcleos físicos: 15 processos ainda cabem um por núcleo, e 31 usam o HT.
3. A versão balanceada (alterando o código) com **2, 4, 8, 16 e 32 processos**, em que 16 é um por núcleo físico e 32 usa o HT, e também com **31**, para comparar diretamente com a versão original e mostrar o que acontece quando P não é potência de 2.
4. A versão original com **63, 127, 255, 511 e 1023 processos**, nos mesmos 2 nós: são mais processos que as 32 threads de hardware, e é isso que põe o escalonador para balancear. Meça esta série duas vezes: com `OMPI_MCA_mpi_yield_when_idle=0` e com `OMPI_MCA_mpi_yield_when_idle=1`, definidos explicitamente.

Para cada caso, calcule dois speed-ups:

- $S = T_1/T_P$, contra o Bubble Sort do vetor inteiro. Pode passar de P, porque inclui o ganho de algoritmo da seção 2.
- $S^\delta = T_1^\delta/T_P$, contra a mesma árvore em um só processo. Mede só o paralelismo, e dele vem a eficiência $E = S^\delta/P$. Na série de 63 a 1023 processos, que passa do número de threads de hardware, calcule a eficiência em relação às 32 threads ($S^\delta/32$).

A razão T_1/T_1^δ é o ganho de algoritmo, e o relatório deve usá-la para explicar o speed-up acima de P.

7. O relatório

Em PDF, com **uma página de análise dos resultados** e o **código nas páginas seguintes**, sem limite de tamanho, em coluna única e com números de linha. Cabeçalho compacto com os integrantes. Toda figura e

tabela tem legenda e é referenciada no texto; passagens importantes do código são referenciadas pelo número da linha. A análise cobre:

1. **Modelagem:** como a árvore é montada, como cada processo descobre pai e filhos, e como a versão balanceada muda isso.
2. **Speed-up e eficiência** das duas versões, separando o ganho de paralelismo (S^{δ}) do ganho de algoritmo (T_1/T_1^{δ}), e a distribuição da carga na versão balanceada com 31 processos.
3. **Balanceamento da carga** nos diferentes níveis da árvore, na versão original.
4. **O ganho obtido com o HT.**
5. **O ganho obtido com o balanceamento**, sem alterar o código (mais processos) e alterando o código.
6. **O desempenho com 63 a 1023 processos**, com espera ativa e com o núcleo cedido, e o que a diferença entre as duas mostra sobre o balanceamento feito pelo escalonador.
7. **Uso de ferramentas de IA**, se houve.

Analisar resultados significa explicar o que aconteceu, não apenas repetir o que se vê nos gráficos.

8. Entrega

No Moodle, até a data do cronograma, um envio por grupo: o relatório em PDF. Os scripts `sbatch` e as saídas do Slurm com os tempos brutos vão em um repositório git **público** (link no cabeçalho) ou em um `.zip` enviado junto do PDF, para que os resultados possam ser reproduzidos.

9. Apresentação

Em sala, para a turma, em **no máximo 10 minutos**, priorizando a modelagem e a análise. Cada integrante precisa estar presente e participar; quem faltar ou não apresentar recebe zero, mesmo que o grupo tenha entregado. Qualquer integrante pode ser questionado sobre qualquer chamada MPI do código e qualquer número do relatório.

10. Avaliação

Cada item é avaliado em quatro níveis: atingiu plenamente, parcialmente, quase não atingiu, não atingiu. A presença na apresentação é condição individual e antecede estes critérios.

#	Item
1	O relatório está bem escrito e no formato solicitado
2	Execução da versão sequencial do algoritmo escolhido
3	Implementação da versão paralela SPMD seguindo o modelo divisão e conquista
4	Medição dos tempos da versão sequencial e da paralela em 2 nós exclusivos
5	Cálculo do speed-up e da eficiência com 1, 3, 7, 15 e 31 processos
6	Análise do balanceamento da carga nos diferentes níveis da árvore
7	Análise do ganho obtido com HT
8	Análise do ganho obtido com a inclusão de um mecanismo de balanceamento de carga
9	Execução com 63, 127, 255, 511 e 1023 processos e análise do desempenho obtido
10	Clareza do código (comentários e nomes de variáveis adequados)
11	Participação do grupo nas aulas de acompanhamento
12	Na apresentação, cada integrante demonstrou domínio do código, dos resultados e respondeu às perguntas

Anexo: pseudocódigo da versão original

Todos os processos executam o mesmo programa. A metade direita fica com `tam_vetor - tam_vetor/2` elementos, para que nenhum se perca quando o tamanho é ímpar.

```
MPI_Init()
my_rank = MPI_Comm_rank()

// recebo o vetor
se my_rank != 0
    MPI_Probe(pai, &status)                        // espero a mensagem do pai
    MPI_Get_count(&status, MPI_INT, &tam_vetor)    // descubro o tamanho recebido
    aloca vetor com tam_vetor posições
    MPI_Recv(vetor, tam_vetor, pai)
senão
    tam_vetor = VETOR_SIZE                         // sou a raiz: gero o vetor
    Inicializa(vetor, tam_vetor)                   // em ordem decrescente

// dividir ou conquistar?
se tam_vetor <= delta
    BubbleSort(vetor, tam_vetor)                   // conquisto
senão
    meio = tam_vetor / 2                           // divido
    MPI_Send(&vetor[0], meio, filho_esquerda)
    MPI_Send(&vetor[meio], tam_vetor - meio, filho_direita)
    MPI_Recv(&vetor[0], meio, filho_esquerda)      // recebo dos filhos
    MPI_Recv(&vetor[meio], tam_vetor - meio, filho_direita)
    Intercala(vetor, tam_vetor)

// mando para o pai
se my_rank != 0
    MPI_Send(vetor, tam_vetor, pai)
senão
    Verifica(vetor)                                // sou a raiz

MPI_Finalize()
```

Errata da versão 2

Duas passagens da primeira versão estavam incorretas. Os requisitos, as medições exigidas e os critérios de avaliação não mudaram.

- **Seção 2, tempos de referência.** Os tempos citados (74 minutos com um processo, 18 com 3, para 1000000 de elementos) não correspondem ao `sequencial.c` compilado com `-O2`. A versão 2 mantém só a proporção: com 3 processos, o tempo cai para cerca de 1/4.

  Para quem já começou: se o tamanho do vetor foi escolhido a partir daqueles tempos, confira se T_1 está entre 30 e 60 segundos (seção 5).

- **Seção 4 e item 4 da seção 6, espera ativa.** A primeira versão dizia que o Open MPI faz espera ativa por padrão e pedia a série de 63 a 1023 processos "com a configuração padrão". Dependendo de como o programa é lançado, o processo já cede o núcleo sem que a variável seja definida, e as duas séries medem a mesma coisa. A versão 2 pede as duas séries com `OMPI_MCA_mpi_yield_when_idle=0` e `OMPI_MCA_mpi_yield_when_idle=1`, definidos explicitamente.

  Para quem já começou: se a série "padrão" foi medida sem definir a variável, refaça essa série com `=0`. A série com `=1` continua válida.